

# 作业：后端开发

## 分工说明

本次作业按角色分工完成，负责后端的同学只需完成后端部分。

## 作业目标

完成项目后端的核心 API 开发、数据库设计，并提供可运行的容器化服务供前端对接。

## 具体要求

### 1. 核心功能实现

- 用户认证 (JWT 推荐)
- CRUD 接口 (至少 2 个业务资源)
- 数据验证和统一错误响应格式
- 数据库操作 (推荐使用 ORM)

统一响应格式 (前后端约定)：

```
{ "code": 0, "data": {}, "msg": "success" }
```

### 2. 代码结构

```
backend/  
├── app/  
│   ├── models/      # 数据模型  
│   ├── routes/      # 路由/控制器  
│   ├── services/    # 业务逻辑  
│   └── utils/        # 工具函数  
├── tests/            # 测试文件  
├── Dockerfile  
└── requirements.txt  # 或 package.json / go.mod
```

### 3. Docker 化部署

部署由后端负责，确保前端可以通过 `docker-compose up` 启动完整后端服务。

```
# Dockerfile 示例 (Python)  
FROM python:3.12-slim  
WORKDIR /app  
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

```
# docker-compose.yml
services:
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/myapp
    depends_on:
      - db
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: myapp
```

## 目录结构

```
project/
├── backend/                                # 后端项目（本次作业范围）
│   ├── app/
│   ├── tests/
│   ├── Dockerfile
│   └── requirements.txt
├── docs/
│   └── contributions/
│       ├── 06-backend/
│       │   ├── zhangsan.md
│       │   └── lisi.md
└── docker-compose.yml
```

## 提交内容

### Git 仓库提交

1. 后端项目完整代码（backend/ 目录）
2. 根目录 docker-compose.yml（含数据库服务）
3. docs/contributions/06-backend/你的名字.md

## 学习通提交

提交以下截图作为个人完成证明：

1. **Git 提交记录** — `git log --author="YOUR_NAME" --oneline` 终端截图
2. **API 测试截图** — Postman/Apifox/Swagger 测试成功的截图（至少 3 个接口）
3. **Docker 运行截图** — `docker-compose ps` 显示服务正常运行

💡 确保 `docker-compose up -d` 可以在组员电脑上启动，前端同学才能对接。

## 提交截止

下次课前

## 个人贡献说明

每位后端成员在 `docs/contributions/06-backend/` 下提交贡献说明：

文件名：你的名字.md

内容模板：

```
# 后端开发贡献说明

姓名: XXX
学号: XXX
日期: XXXX-XX-XX

## 我完成的工作

### API 实现
- [ ] 用户认证 API（注册 / 登录）
- [ ] 业务资源 1 CRUD: XXX
- [ ] 业务资源 2 CRUD: XXX
- [ ] 统一错误响应

### 数据库
- [ ] 数据模型定义（ER 图或模型文件）
- [ ] ORM 配置
- [ ] 数据库迁移脚本

### 部署
- [ ] Dockerfile 编写
- [ ] docker-compose.yml 配置
- [ ] 本地联调验证
```

## ## PR 链接

- PR #X: <https://github.com/xxx/xxx/pull/X>

## ## 遇到的问题和解决

1. 问题: XXX  
解决: XXX

## ## 心得体会

(简要说明后端开发过程中的收获)

## 评分标准

| 项目        | 分值  | 说明                                   |
|-----------|-----|--------------------------------------|
| API 完成度   | 35% | 至少 2 个资源 CRUD + 认证接口                 |
| 代码结构      | 20% | 模块划分清晰, 错误处理完善                       |
| Docker 部署 | 25% | <code>docker-compose up</code> 可一键启动 |
| 个人贡献说明    | 20% | 贡献描述清晰, 有截图证明                        |

## 额外加分项

- 使用 ORM 并提交迁移文件
- 实现 JWT 认证
- 编写 API 测试用例 (pytest / Jest)
- 配置 CORS, 前后端成功联调